

Multi-Stage CI/CD Pipeline

Online Bookstore Application

Complete Project Documentation

Technologies Used

AWS EC2 | Jenkins | GitHub | Docker | Docker Hub | Kubernetes | Maven

Author: Pavan (pavan7302) | Date: March 2026

Section 1: What Is This Project? (Simple Explanation)

Imagine You Are a Baker

Imagine you own a bakery. Every day you bake cakes (your software), put them in a box (Docker container), and deliver them to customers (users). Now imagine if this ENTIRE process happened AUTOMATICALLY every time you changed your recipe (your code). That is exactly what this project does - but for software!

The Simple Story

We built an Online Book Store website. But the REAL achievement is the MAGICAL CONVEYOR BELT that automatically:

- Takes your new code from GitHub (like a recipe change)
- Builds it using Maven (like mixing all ingredients)
- Packages it into Docker (like putting the cake in a box)
- Uploads it to Docker Hub (like storing the boxed cake in a warehouse)
- Deploys it to Kubernetes (like delivering cakes to ALL customers automatically)

And the BEST part? You just push code to GitHub and EVERYTHING ELSE HAPPENS BY ITSELF in under 3 minutes!

Manual vs Automated Comparison

Without This Project (Manual)	With This Project (Automated)
Developer writes code	Developer writes code
MANUALLY runs build commands	Jenkins automatically builds it
MANUALLY creates Docker image	Jenkins creates Docker image
MANUALLY uploads to server	Jenkins pushes to Docker Hub
MANUALLY deploys to servers	Kubernetes deploys automatically
Takes 1-2 hours, error-prone	Takes under 3 minutes, zero errors
Someone has to be awake at 3am	Happens 24/7 without anyone watching

Section 2: Understanding Each Technology

1. AWS EC2 - Our Computer in the Cloud

```
Think of EC2 like: Renting a computer at a data center
Instance type: t2.medium (2 vCPU, 4GB RAM, 20GB storage)
OS: Ubuntu 22.04 LTS
Region: Asia Pacific - Mumbai (ap-south-1)
```

Instead of buying a physical computer, we rent one from Amazon. It lives in Amazon's data center in Mumbai, is always on, always connected to the internet, and we pay only for what we use.

2. GitHub - Our Code Safe Vault

```
Think of GitHub like: Google Drive but specifically for code
Stores all our code and tracks every change ever made
Repo: github.com/pavan7302/onlinebookstore
Special power: Sends a webhook to Jenkins whenever code is pushed
```

3. Jenkins - Our Robot Manager

```
Think of Jenkins like: A very obedient robot that never sleeps
Receives the webhook from GitHub and runs all 5 pipeline stages
Running as: WAR file (nohup java -jar jenkins.war)
```

URL: `http://13.233.232.229:8080`

4. Maven - Our Build Tool

Think of Maven like: A master chef who knows exactly how to cook the recipe
Downloads all required libraries, compiles Java code, creates WAR file
Command used: `mvn clean package -DskipTests`
Output: `bookstore.war` (the packaged application)

5. Docker - Our Shipping Container

Think of Docker like: A magical box containing EVERYTHING the app needs
Packages the app + Java + Tomcat server into one portable container
Our image: `pavan7302/bookstore:latest`
Base image: `tomcat:9.0-jdk17`

6. Docker Hub - Our Container Warehouse

Think of Docker Hub like: Amazon warehouse for Docker containers
Stores and serves Docker images to anyone who needs them
Our images: `pavan7302/bookstore:1`, `:2`, `:3` (one per build)
URL: `hub.docker.com`

7. Kubernetes (Minikube) - Our Army of Servers

Think of Kubernetes like: A very smart manager who runs your app on multiple servers
Runs 2 copies of the app, restarts if one crashes, updates without downtime
Setup: Minikube (single-node K8s) running inside Docker on EC2
Key feature: Rolling updates = zero downtime deployments

Section 3: The 5-Stage Pipeline Explained

Stage 1: Source Checkout (0.85 seconds)

Jenkins downloads the latest code from GitHub
Command: `checkout scm`
Result: All project files available in Jenkins workspace

Jenkins connects to GitHub and downloads ALL the files in your repository. Like a student downloading homework from Google Classroom before starting work.

Stage 2: Maven Build (15 seconds)

```
Maven compiles Java code and creates a WAR file
Command: mvn clean package -DskipTests
Input: Java source files (.java) | Output: bookstore.war
```

- Downloads all required Java libraries (first run only)
- Compiles all .java files into .class files (machine-readable)
- Packages everything into a single bookstore.war file

Stage 3: Docker Image Build (11 seconds)

```
Jenkins builds a Docker image containing the app
Command: docker build -t pavan7302/bookstore:BUILD_NUMBER .
Input: bookstore.war + Dockerfile | Output: Docker image
```

Each build gets a unique tag: pavan7302/bookstore:1, :2, :3 ... The 'latest' tag always points to the most recent build.

Stage 4: Push to Docker Hub (19 seconds)

```
The Docker image is uploaded to Docker Hub registry
Command: docker push pavan7302/bookstore:BUILD_NUMBER
Authentication: Jenkins credential 'dockerhub-creds' (password never visible in logs)
```

Stage 5: Deploy to Kubernetes (16 seconds)

```
Kubernetes updates the running application with the new version
Commands: kubectl apply -f deployment.yaml, kubectl set image, kubectl rollout status
Strategy: RollingUpdate (maxSurge:1, maxUnavailable:0) = zero downtime
```

1. Kubernetes sees the new image tag
2. Starts ONE new pod with new version (3 pods: 2 old + 1 new)
3. Once new pod is healthy, removes ONE old pod (2: 1 old + 1 new)
4. Repeats until all pods run the new version
5. Users NEVER experience downtime - at least 1 pod always running

Section 4: Every Project File Explained

Jenkinsfile

Tells Jenkins exactly what to do. Located in the GitHub repo root. Written in Groovy language.

- environment: DOCKER_IMAGE, IMAGE_TAG, KUBECONFIG variables
- tools: Uses Maven3 for building
- stages: All 5 pipeline stages in order
- post: Runs docker logout after every build

Dockerfile

```
FROM tomcat:9.0-jdk17          # Start with Tomcat + Java 17
RUN rm -rf /usr/local/tomcat/webapps/*    # Remove default apps
COPY target/bookstore.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080                          # App runs on port 8080
CMD ["catalina.sh", "run"]          # Start Tomcat server
```

deployment.yaml

Deployment section:

- 2 replicas (copies) always running
- RollingUpdate with maxUnavailable:0 (zero downtime)
- imagePullPolicy: Always (always fetch latest)

Service section:

- NodePort 30080 for external access
- Routes traffic: external port 80 -> container port 8080

pom.xml

- Project: online-bookstore, version 1.0-SNAPSHOT
- Package type: WAR (for Tomcat server)
- Dependency: javax.servlet-api
- Build output: bookstore.war

Section 5: System Architecture

Complete Flow Diagram

```
DEVELOPER
  |  git push
  v
GITHUB (pavan7302/onlinebookstore - master branch)
  |  Webhook notification
  v
JENKINS (EC2 port 8080)
  |-- Stage 1: git checkout
  |-- Stage 2: mvn clean package --> bookstore.war
  |-- Stage 3: docker build      --> Docker Image
  |-- Stage 4: docker push       --> Docker Hub
  |-- Stage 5: kubectl apply     --> Kubernetes
  v
DOCKER HUB (pavan7302/bookstore:BUILD_NUMBER)
  |  Kubernetes pulls image
  v
KUBERNETES MINIKUBE (same EC2)
  |-- Pod 1: bookstore (Running)
  |-- Pod 2: bookstore (Running)
  v
USERS: http://13.233.232.229:8081
```

Infrastructure Details

Component	Where It Runs	Port
Jenkins WAR	EC2 (ubuntu user, nohup)	8080
Minikube	EC2 (Docker driver)	Internal
Bookstore App	Minikube pods x2	8080 internal
Port-forward	EC2 kubectl process	8081 -> 80
Docker	EC2 system service	Unix socket
GitHub Webhook	GitHub -> EC2	8080

Section 6: Detailed Steps to Build This Project from Scratch

Follow every step below in order. Do not skip any step. Each step builds on the previous one.

Phase 1: Create AWS EC2 Instance

1 Log in to AWS Console

Go to <https://console.aws.amazon.com> and sign in with your AWS account.

2 Navigate to EC2

Click the search bar at the top, type EC2, and click on EC2 under Services.

3 Launch a New Instance

- Click the orange 'Launch instances' button
- Name: CI-CD-Server
- AMI: Ubuntu Server 22.04 LTS (Free tier eligible)
- Instance type: t2.medium (2 vCPU, 4 GiB RAM) - IMPORTANT: Do NOT use t2.micro
- Key pair: Create new key pair, name it 'cicd-key', type RSA, format .pem, Download it

4 Configure Security Group

In the 'Network settings' section, click 'Edit' and add these inbound rules:

Type	Port	Source
SSH	22	0.0.0.0/0
Custom TCP	8080	0.0.0.0/0 (Jenkins)
Custom TCP	8081	0.0.0.0/0 (App port-forward)
Custom TCP	30080	0.0.0.0/0 (Kubernetes NodePort)

5 Launch and Connect

- Click 'Launch instance'
- Wait 2 minutes for the instance to start
- Click on the instance ID, then click 'Connect'
- Use 'EC2 Instance Connect' tab, click 'Connect'
- A browser terminal window opens - you are now inside your EC2 server

NOTE: If you see 'Pending kernel upgrade' dialog - press Enter to dismiss it, then run: `sudo reboot`
Wait 30 seconds after reboot, then reconnect via EC2 Instance Connect

Phase 2: Install Java

6 Update the system

```
sudo apt update
sudo apt upgrade -y
```

7 Install Java 17

```
sudo apt install -y openjdk-17-jdk
```

8 Verify Java installation

```
java -version
# Expected output: openjdk version 17.x.x
```

Phase 3: Install Jenkins (WAR file method)

9 Download Jenkins WAR file

```
wget https://get.jenkins.io/war-stable/latest/jenkins.war
ls -lh jenkins.war
# Should show a file around 90MB
```

10 Start Jenkins in background

```
nohup java -jar ~/jenkins.war --httpPort=8080 > ~/jenkins.log 2>&1 &

# nohup = keeps running even after you close the terminal
# & = runs in background so you can still type commands
```

11 Wait for Jenkins to fully start

```
tail -f ~/jenkins.log

# Wait until you see this line:
# Jenkins is fully up and running

# Press Ctrl+C to stop following the log
# Jenkins will keep running in the background
```

12 Get the initial admin password

```
cat ~/.jenkins/secrets/initialAdminPassword
# Copy this password - you will need it in the browser
```

13 Open Jenkins in your browser


```
# Open this URL in Chrome/Firefox:
# http://YOUR_EC2_PUBLIC_IP:8080

# Get your public IP from EC2 console dashboard
```

When the Jenkins unlock page appears:

6. Paste the admin password you copied in Step 12
7. Click 'Continue'
8. Click 'Install suggested plugins' and wait 5 minutes
9. Create admin user: fill in username, password, email
10. Click 'Save and Continue' -> 'Save and Finish' -> 'Start using Jenkins'

Phase 4: Install Docker

14 Install Docker

```
sudo apt install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker
```

15 Add ubuntu user to docker group

```
sudo usermod -aG docker ubuntu
newgrp docker
```

16 Verify Docker works

```
docker --version
docker ps
# Both commands should work without 'sudo'
```

Phase 5: Install Maven

17 Install Maven

```
sudo apt install -y maven
```

18 Verify Maven

```
mvn --version
# Expected: Apache Maven 3.x.x
```

Phase 6: Install kubectl and Minikube

19 Install kubectl

```
curl -LO "https://dl.k8s.io/release/$(curl -sL
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubect1"
sudo install -o root -g root -m 0755 kubect1 /usr/local/bin/kubect1
kubect1 version --client
```

20 Download Minikube (use exact version - NOT latest)

```
curl -LO https://storage.googleapis.com/minikube/releases/v1.32.0/minikube-linux-
amd64

# Verify it downloaded correctly (must say ELF binary, not XML):
file minikube-linux-amd64
# CORRECT: 'ELF 64-bit LSB executable'
# WRONG: 'XML document' = download failed, try wget instead
```

21 Install Minikube

```
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
# Expected: minikube version: v1.32.0
```

22 Start Minikube

```
minikube start --driver=docker --memory=2200mb --cpus=2

# This takes 3-5 minutes on first run (downloads images)
# Wait until you see:
# Done! kubect1 is now configured to use 'minikube' cluster
```

23 Verify Minikube and cluster

```
minikube status
# Should show: host: Running, kubelet: Running, apiserver: Running

kubect1 get nodes
# Should show: minikube    Ready    control-plane
```

Phase 7: Set Up GitHub Repository

24 Fork or create the repository on GitHub

Option A - Use existing forked repo (what we did):

- Go to github.com/pavan7302/onlinebookstore
- This repo already has the Java bookstore application code

Option B - Create from scratch:

- Go to github.com -> New repository
- Name: online-bookstore, Public, NO README

25 Clone the repo on EC2

```
cd ~
git clone https://github.com/YOUR_USERNAME/onlinebookstore.git
cd onlinebookstore
ls
```

26 Create the Dockerfile

```
cat > Dockerfile << 'EOF'
FROM tomcat:9.0-jdk17
RUN rm -rf /usr/local/tomcat/webapps/*
COPY target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
EOF
```

27 Create deployment.yaml

Replace YOUR_DOCKERHUB_USERNAME with your actual Docker Hub username:

```
cat > deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bookstore-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: bookstore
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: bookstore
    spec:
      containers:
      - name: bookstore
        image: YOUR_DOCKERHUB_USERNAME/bookstore:latest
        ports:
        - containerPort: 8080
        imagePullPolicy: Always
---
apiVersion: v1
kind: Service
metadata:
  name: bookstore-service
spec:
  selector:
    app: bookstore
```

```

    type: NodePort
    ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080
EOF

```

28 Create Jenkinsfile

Replace YOUR_DOCKERHUB_USERNAME with your actual Docker Hub username:

```

cat > Jenkinsfile << 'EOF'
pipeline {
    agent any
    environment {
        DOCKERHUB_CREDENTIALS = credentials('dockerhub-creds')
        DOCKER_IMAGE = 'YOUR_DOCKERHUB_USERNAME/bookstore'
        IMAGE_TAG = "${BUILD_NUMBER}"
        KUBECONFIG = '/home/ubuntu/.kube/config'
    }
    tools { maven 'Maven3' }
    stages {
        stage('Stage 1 - Source Checkout') {
            steps { checkout scm }
        }
        stage('Stage 2 - Maven Build') {
            steps { sh 'mvn clean package -DskipTests' }
        }
        stage('Stage 3 - Docker Image Build') {
            steps {
                sh '''
                    docker build -t ${DOCKER_IMAGE}:${IMAGE_TAG} .
                    docker tag ${DOCKER_IMAGE}:${IMAGE_TAG}
                    ${DOCKER_IMAGE}:latest
                '''
            }
        }
        stage('Stage 4 - Push to Docker Hub') {
            steps {
                sh '''
                    echo ${DOCKERHUB_CREDENTIALS_PSW} | docker login -u
                    ${DOCKERHUB_CREDENTIALS_USR} --password-stdin
                    docker push ${DOCKER_IMAGE}:${IMAGE_TAG}
                    docker push ${DOCKER_IMAGE}:latest
                '''
            }
        }
        stage('Stage 5 - Deploy to Kubernetes') {
            steps {
                sh '''
                    kubectl apply -f deployment.yaml
                    kubectl set image deployment/bookstore-deployment
                    bookstore=${DOCKER_IMAGE}:${IMAGE_TAG}
                    kubectl rollout status deployment/bookstore-deployment
                '''
            }
        }
    }
}

```

```

    }
  }
  post {
    always { sh 'docker logout' }
    success { echo 'Pipeline succeeded! Bookstore is live.' }
    failure { echo 'Pipeline failed. Check console output.' }
  }
}
EOF

```

29 Push all files to GitHub

```

git add Dockerfile deployment.yaml Jenkinsfile
git commit -m 'Add CI/CD pipeline files'
git push origin master

# When prompted:
# Username: your GitHub username
# Password: your Personal Access Token (NOT your GitHub password)

# To create a token: GitHub -> Settings -> Developer settings
#   -> Personal access tokens -> Tokens (classic) -> Generate new token
#   -> Check 'repo' scope -> Generate -> Copy the token

```

Phase 8: Configure Jenkins

30 Install required Jenkins plugins

- In Jenkins: Manage Jenkins -> Plugins -> Available plugins
- Search and install (one by one): Git, GitHub Integration, Pipeline, Docker Pipeline
- Click 'Install without restart'

31 Configure Maven in Jenkins

- Go to: Manage Jenkins -> Tools -> Maven installations
- Click 'Add Maven'
- Name: Maven3 (must be exactly 'Maven3' - matches the Jenkinsfile)
- Check: Install automatically
- Version: 3.9.6
- Click Save

32 Add Docker Hub credentials

- Go to: Manage Jenkins -> Credentials -> System -> Global credentials -> Add Credentials

Field	Value to Enter
Kind	Username with password
Scope	Global

Username	your Docker Hub username (e.g. pavan7302)
Password	your Docker Hub password or access token
ID	dockerhub-creds (EXACTLY this - no spaces)
Description	Docker Hub credentials

Click Create. The ID 'dockerhub-creds' must exactly match what is written in the Jenkinsfile.

Phase 9: Create the Jenkins Pipeline Job

33 Create new pipeline job

11. Click 'New Item' in the left sidebar
12. Enter name: bookstore-pipeline
13. Select 'Pipeline'
14. Click OK

34 Configure the pipeline

15. Scroll down to the 'Pipeline' section
16. Definition: Pipeline script from SCM
17. SCM: Git
18. Repository URL: https://github.com/YOUR_USERNAME/onlinebookstore.git
19. Branch Specifier: */master
20. Script Path: Jenkinsfile

35 Enable GitHub webhook trigger

- In the same job configuration page, scroll up to 'Build Triggers'
- Check the box: 'GitHub hook trigger for GITScm polling'
- Click Save

36 Run the first build manually

- Click 'Build Now' in the left sidebar
- A new build appears under 'Build History'
- Click the build number (e.g. '#1')
- Click 'Console Output' to watch the pipeline run
- Wait 2-3 minutes for all 5 stages to complete

EXPECTED RESULT: You should see all 5 green checkmarks:

Stage 1 - Source Checkout: 0.85s
 Stage 2 - Maven Build: ~15s
 Stage 3 - Docker Image Build: ~11s
 Stage 4 - Push to Docker Hub: ~19s

Stage 5 - Deploy to Kubernetes: ~16s
Final message: Pipeline succeeded! Bookstore is live.

Phase 10: Access the Live Application

37 Verify pods are running

```
kubectl get pods
# Expected:
# bookstore-deployment-xxxxx-xxxxx 1/1 Running 0 2m
# bookstore-deployment-xxxxx-xxxxx 1/1 Running 0 2m

kubectl get services
# Expected: bookstore-service NodePort ...
```

38 Set up port-forwarding to access app in browser

```
kubectl port-forward service/bookstore-service 8081:80 --address 0.0.0.0 &

# You should see: Forwarding from 0.0.0.0:8081 -> 8080

# Verify it is listening:
ss -tlnp | grep 8081
```

39 Open the application in your browser

```
# Open this URL in Chrome:
http://YOUR_EC2_PUBLIC_IP:8081

# You should see: 'Welcome to Online Book Store'
# With Home, Login, Register navigation menu
```

Phase 11: Set Up GitHub Webhook (Auto-trigger on every push)

40 Add webhook in GitHub

21. Go to your GitHub repository
22. Click Settings (top menu)
23. Click Webhooks (left sidebar)
24. Click 'Add webhook'

Field	Value
Payload URL	http://YOUR_EC2_PUBLIC_IP:8080/github-webhook/
Content type	application/json
Which events	Just the push event

Active	Checked (enabled)
--------	-------------------

25. Click 'Add webhook'

26. GitHub will show a green tick if it successfully pinged Jenkins

41 Test the complete automation

```
# Make any small change to the code:
cd ~/onlinebookstore
echo "<!-- test -->" >> README.md
git add .
git commit -m "Test webhook auto-trigger"
git push origin master

# Now watch Jenkins dashboard at http://YOUR_EC2_IP:8080
# A new build should start automatically within seconds!
```

SUCCESS CRITERIA - Your project is complete when:

1. git push automatically triggers Jenkins build
2. All 5 stages complete with green checkmarks
3. New Docker image appears in Docker Hub
4. kubectl get pods shows 2 Running pods
5. Browser shows the Online Book Store at port 8081

Section 7: Problems Faced and Solutions

Problem Encountered	How We Fixed It
Jenkins GPG key error during apt install	Used Jenkins WAR file method instead
Minikube download returned XML error	Used explicit version URL v1.32.0 instead of latest
RSRC_INSUFFICIENT_CONTAINER_MEMORY error	Upgraded EC2 to t2.medium (4GB RAM)
Port 8080 connection refused in browser	Added inbound rule in EC2 Security Group
chown: invalid user jenkins:jenkins	Jenkins WAR runs as ubuntu user, not jenkins user
kubectl current-context is not set	Ran minikube start to initialize cluster
Browser timeout on port 8081	Fixed Security Group + ran kubectl port-forward
Pending kernel upgrade dialog	Pressed Enter, ran sudo reboot
git push: repository not found	Created the repo on GitHub first, then pushed
docker: permission denied	Added ubuntu user to docker group, ran newgrp docker

Section 8: Key Achievements and Metrics

Achievement	Details
70% Release Cycle Reduction	From ~60 minutes manual to ~3 minutes automated
Zero-Downtime Deployment	maxUnavailable:0 ensures no downtime during updates
100% Automated Deployment	Zero manual steps from code push to live app
Stage 1 - Source Checkout	0.85 seconds
Stage 2 - Maven Build	15 seconds
Stage 3 - Docker Image Build	11 seconds
Stage 4 - Docker Hub Push	19 seconds
Stage 5 - Kubernetes Deploy	16 seconds
Total Pipeline Time	Under 3 minutes end to end
High Availability	2 pods always running, auto-restart on crash
Rollback Capability	kubect! rollout undo (instant rollback)
Image Versioning	Every build tagged with unique build number

Section 9: Commands Cheat Sheet

Jenkins Commands

```
# Start Jenkins
nohup java -jar ~/jenkins.war --httpPort=8080 > ~/jenkins.log 2>&1 &

# Check Jenkins logs
tail -f ~/jenkins.log

# Check if Jenkins is running
ps aux | grep jenkins

# Get initial admin password
cat ~/.jenkins/secrets/initialAdminPassword
```

Minikube Commands

```
minikube start --driver=docker --memory=2200mb --cpus=2
minikube status
minikube stop
minikube delete # WARNING: deletes entire cluster
```

Kubernetes Commands

```
kubectl get pods           # See running pods
kubectl get deployments    # See deployments
kubectl get services       # See services
kubectl logs <pod-name>    # See app logs
kubectl rollout status deployment/bookstore-deployment
kubectl rollout undo deployment/bookstore-deployment # Rollback!
kubectl describe pod <pod-name> # Detailed pod info
```

Access the Application

```
# Port-forward to access app in browser
kubectl port-forward service/bookstore-service 8081:80 --address 0.0.0.0 &

# Verify it is working
curl http://localhost:8081

# Then open: http://YOUR_EC2_PUBLIC_IP:8081
```

Section 10: Glossary

Term	Simple Explanation
CI/CD	Continuous Integration/Continuous Deployment - automates building, testing and deploying software
Pipeline	A series of automated steps that run one after another, like an assembly line
Repository	A storage place for code on GitHub that tracks all changes ever made
Webhook	An automatic notification sent from one service to another when something happens
Container	A package with an app + everything it needs to run, like a sealed lunchbox
Docker Image	A frozen template used to create containers, like a cake mold
Docker Hub	An online warehouse where Docker images are stored and shared
Kubernetes	A system that manages containers, keeps them running, and updates them
Pod	The smallest unit in Kubernetes - one running container

Deployment	A Kubernetes object ensuring a certain number of pods always run
Service	A Kubernetes object that exposes pods to the outside world
NodePort	A way to access a Kubernetes service from outside using a specific port
Rolling Update	Updating pods one at a time so the app is never fully down
WAR file	Web Application Archive - a packaged Java web app ready to run on Tomcat
Maven	A build tool for Java that downloads dependencies and compiles code
Tomcat	A web server that runs Java web applications
EC2	Elastic Compute Cloud - Amazon's service for renting virtual computers
Security Group	AWS firewall rules controlling which ports receive internet traffic
Replica	A copy of a running pod - 2 replicas means 2 identical pods running
nohup	A Linux command that keeps a process running after you close the terminal
Port-forward	Tunneling traffic from EC2 to a Kubernetes service for browser access
Webhook	Automatic HTTP POST sent by GitHub to Jenkins to trigger builds
Credential	Username/password stored securely in Jenkins - never shown in logs
Build Number	Unique number Jenkins assigns to each build - used as Docker image tag

End of Documentation

Multi-Stage CI/CD Pipeline | Online Bookstore | March 2026